FINAL-YEAR PROJECT REPORT

AI-Powered Multilingual Vision Assistant for Visually Impaired People

A real-time, camera-based assistive system that detects people, vehicles, and obstacles and speaks safe navigation guidance in multiple languages.

Python · Flask | OpenCV · YOLOv8 | Multilingual Text-to-Speech

Project Information

Project Title	AI-Powered Multilingual Vision Assistant for Visually Impaired People
Category	Assistive Technology / Computer Vision / Human-Centered AI
Core Technologies	Python, Flask, OpenCV, Ultralytics YOLOv8, pyttsx3, gTTS, googletrans
Interface	Web dashboard (HTML, CSS, JavaScript) served by Flask
Input Source	Laptop webcam (default) or phone IP-camera stream
Output	Annotated live video, on-screen alerts, and spoken voice instructions

Abstract

Visually impaired individuals face persistent challenges in perceiving their immediate surroundings, particularly moving obstacles such as pedestrians and vehicles. This project presents an AI-powered vision assistant that turns any laptop webcam or phone camera into a real-time safety companion. The system captures a live video stream, runs it through a YOLOv8 object-detection model, and classifies the scene into danger levels — safe, low, medium, or high — based on the type, position, and estimated distance of nearby objects.

Once a risk is identified, the system converts the detection into a short, human-readable instruction (for example, "car detected on your right, maintain safe distance") and speaks it aloud using an offline or online text-to-speech engine. Voice output is available in more than a dozen languages, including English, Hindi, Telugu, Tamil, and others, making the tool accessible to a linguistically diverse set of users. All meaningful detections are logged for later review, and the entire experience is presented through a lightweight, glass-styled Flask web dashboard that displays the live annotated camera feed, the current spoken instruction, and the list of detected objects side by side.

1. Introduction

According to global health estimates, hundreds of millions of people live with some form of visual impairment, and a significant fraction of them have little or no functional vision. Independent mobility — walking through a street, a corridor, or an unfamiliar room — remains one of the most difficult everyday tasks for this population. Traditional aids such as the white cane or guide dogs are effective but limited: they can detect obstacles only within arm's or nose's length and provide no information about moving hazards like an oncoming vehicle or a fast-approaching cyclist.

Recent progress in computer vision and deep learning has made real-time object detection fast and accurate enough to run on consumer-grade hardware. This project leverages that progress to build a practical, low-cost assistive system. Instead of specialized hardware, it uses a standard laptop webcam or a smartphone acting as an IP camera, combined with the YOLOv8 object detector, to identify people, vehicles, and common obstacles in the user's path. The detected information is translated into simple spoken guidance, allowing the user to react to their surroundings without needing to look at a screen.

The system was designed with three guiding principles: **real-time performance** (the feedback loop must be fast enough to be useful while walking), **simplicity of guidance** (instructions must be short, clear, and immediately actionable), and **accessibility** (guidance should be available in the user's native language wherever possible).

1.1 Problem Statement

Visually impaired individuals lack an affordable, real-time way to be alerted about approaching people, vehicles, and obstacles in their path, especially in outdoor or semi-structured environments where hazards can appear from any direction and at varying distances.

1.2 Objectives

- Detect people, vehicles, and common obstacles in real time using a standard camera feed.
- Estimate the relative direction (left, center, right) and approximate distance (far, medium, close, very close) of each detected object.
- Classify the overall situation into a danger level and generate a short, actionable instruction.
- Deliver that instruction as spoken audio in the user's preferred language, with minimal delay.
- Present a simple, accessible web dashboard for configuration (language, voice mode, alerts on/off) and for sighted assistants or developers to monitor the system.
- Maintain a persistent log of medium- and high-risk detections for later review.

1.3 Scope

The current version focuses on general-purpose obstacle awareness using a pretrained YOLOv8n model (80 everyday object classes, including person, car, bus, truck, motorcycle, and bicycle). It is designed to run on a single machine — a laptop or a small edge device — and does not currently include GPS navigation, custom-trained hazard classes (potholes, open drains, stairs), or dedicated wearable hardware; these are outlined as future scope.

2. System Architecture

The application follows a simple, modular pipeline built around a Flask backend. Each stage of the pipeline is handled by a dedicated Python module, which keeps detection logic, safety/navigation logic, and voice output cleanly separated.

Module	Responsibility
`app.py`	Flask application entry point. Opens the camera, streams annotated frames to the browser via `/video_feed`, exposes REST endpoints

	for settings, status, and voice testing, and writes risk events to the log file.
`detector.py`	Wraps the Ultralytics YOLOv8 model. Runs inference on each frame, draws bounding boxes and zone lines, and calls the danger logic module to produce a spoken instruction and on-screen banner.
`danger_logic.py`	Pure logic module (no camera or model dependency) that converts a list of detections into a danger level (safe / low / medium / high) and a natural-language instruction, based on object type, screen zone, and estimated distance.
`voice.py`	Handles text-to-speech. Supports an offline engine (pyttsx3) and an online multilingual engine (Google Translate + gTTS) across 14+ languages, with a cooldown to avoid repeating the same alert too frequently.
`templates/index.html` + `static/`	Front-end dashboard: live video panel, voice/language settings, current instruction panel, and detected-objects list, styled as a dark glass-morphism UI.

2.1 Processing Pipeline

Frame capture — OpenCV reads a frame from the webcam (or an IP-camera URL supplied via the `CAMERA_URL` environment variable).

Object detection — The frame is passed to the YOLOv8n model, which returns bounding boxes, class labels, and confidence scores for every detected object above a 0.45 confidence threshold.

Zone and distance estimation — Each detection's horizontal position is mapped to a left / center / right zone (the frame is divided into thirds), and its bounding-box area relative to the frame area is used to approximate distance (far, medium distance, close, very close).

Risk scoring — Objects are scored higher if they belong to the "dangerous" category (cars, buses, trucks, motorcycles, bicycles, trains) and if they fall in the center zone. The highest-scoring object drives the final instruction.

Instruction generation — The danger logic module produces one of four danger levels and a short instruction, e.g. "*Warning. car is very close in front of you. Stop and move carefully.*"

Voice output — If voice alerts are enabled, the instruction is translated (for online mode) and spoken aloud, respecting a cooldown period so the same message is not repeated every frame.

Dashboard update — The annotated frame, current instruction, translated instruction, and detected-object list are streamed to the web dashboard in real time.

Logging — Any medium- or high-risk event is appended to `logs/detections.jsonl` for later analysis.

2.2 Danger Classification Logic

Level	Trigger Condition	Example Instruction
Safe	No relevant objects detected	"Path is clear. Go straight."
Low	Obstacle detected on the far left or right, away from the walking path	"Person detected on your left. Keep slightly right."
Medium	Obstacle ahead but not at close range, or a vehicle off to the side	"Car detected on your right. Maintain safe distance."
High	A person or vehicle is centered and at close / very close range	"Warning. car is very close in front of you. Stop and move carefully."

3. Technology Stack

Backend & AI

- Python 3
- Flask — web server and REST endpoints
- OpenCV — video capture and frame annotation
- Ultralytics YOLOv8 (yolov8n) — object detection
- NumPy — numerical operations on frames

Voice & Language

- pyttsx3 — offline text-to-speech
- gTTS (Google Text-to-Speech) — online multilingual voice
- googletrans — automatic instruction translation

Frontend

- HTML5, CSS3 (glass-morphism dark theme)
- Vanilla JavaScript for settings, polling, and voice test controls

Data & Storage

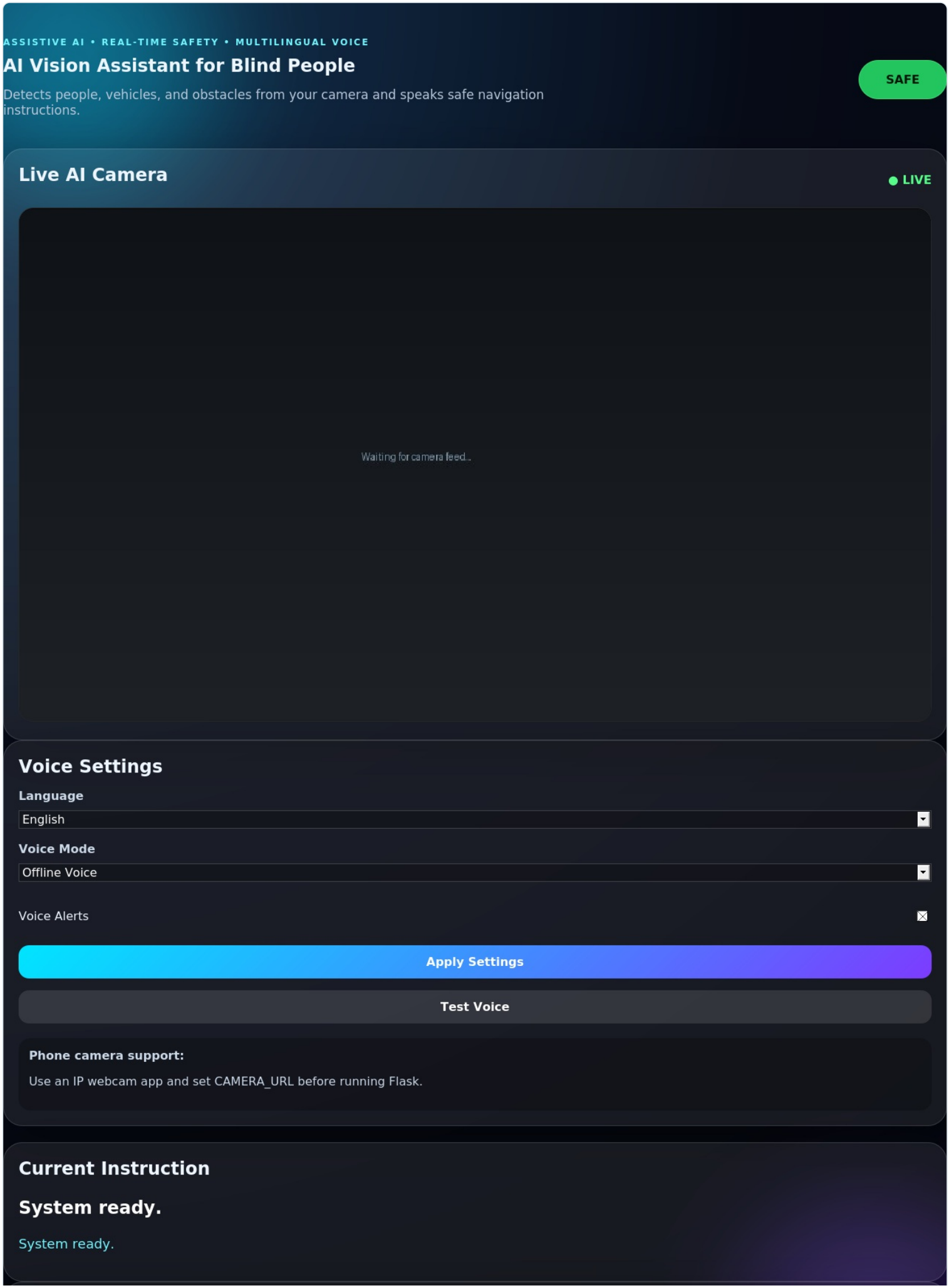
- JSON Lines log file (`logs/detections.jsonl`) for medium/high-risk events
- PyMongo & python-dotenv included for optional future database integration

3.1 Multilingual Support

The voice module ships with built-in support for 14 languages out of the box — English, Telugu, Hindi, Marathi, Gujarati, Tamil, Kannada, Malayalam, Bengali, Urdu, Arabic, French, Spanish, and German — with the language list extendable through a simple dictionary of language-name-to-language-code mappings. Offline voice mode works without an internet connection but is generally limited to the voices installed on the host system (often English only), while Online Multilingual Voice mode uses Google's translation and speech services to produce accurate native-language audio.

4. Application Walkthrough & Screenshots

The following screenshots show the Flask web dashboard in its four main operating states: system idle, a low/medium-risk "caution" detection, a high-risk "danger" detection with multiple objects, and the settings panel configured for a different language and voice mode.



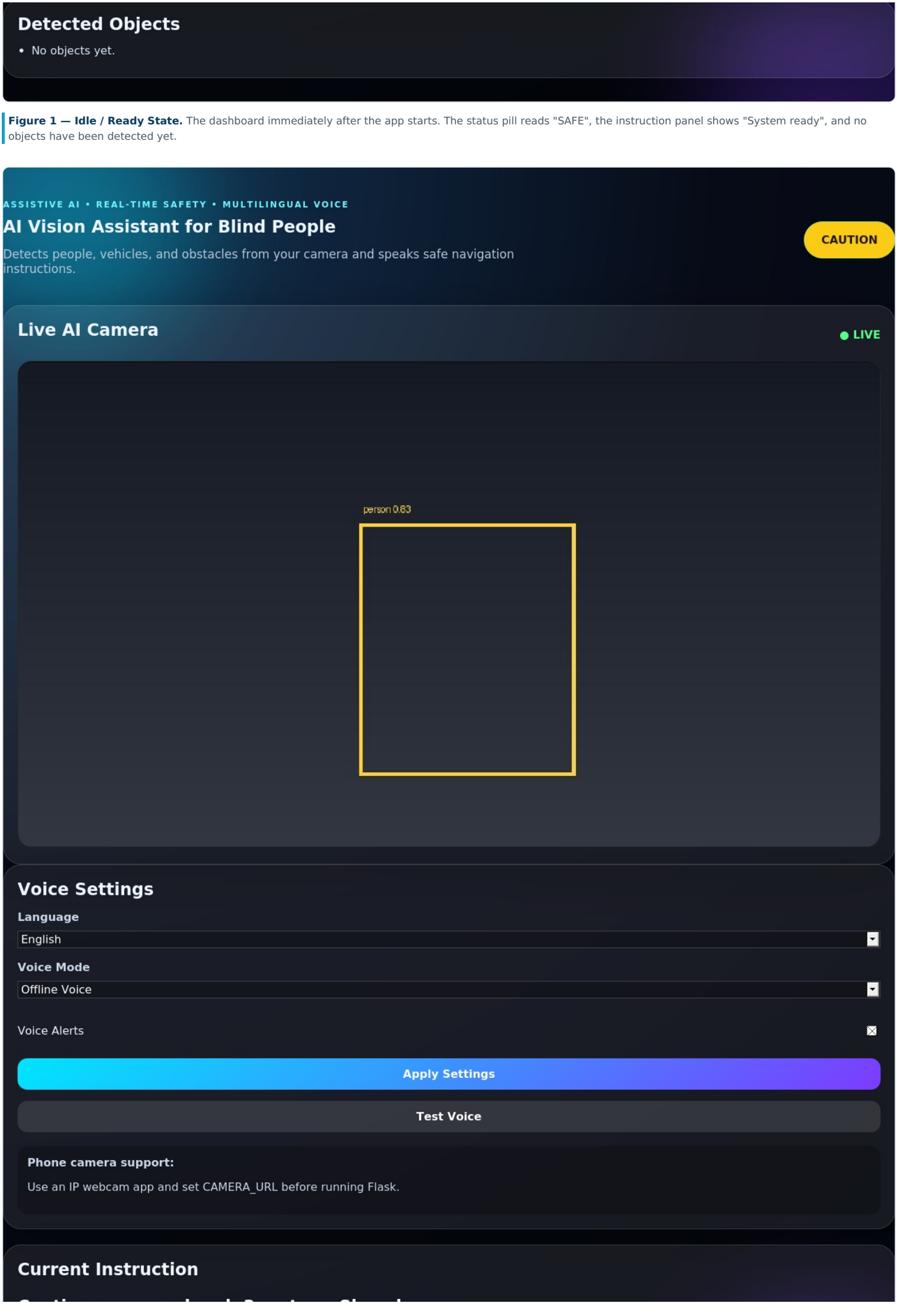


Figure 1 — Idle / Ready State. The dashboard immediately after the app starts. The status pill reads "SAFE", the instruction panel shows "System ready", and no objects have been detected yet.

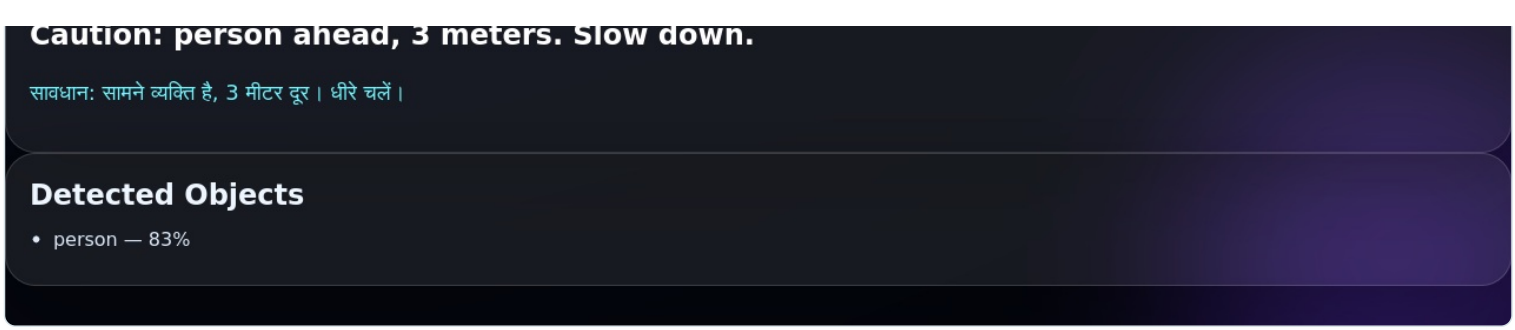
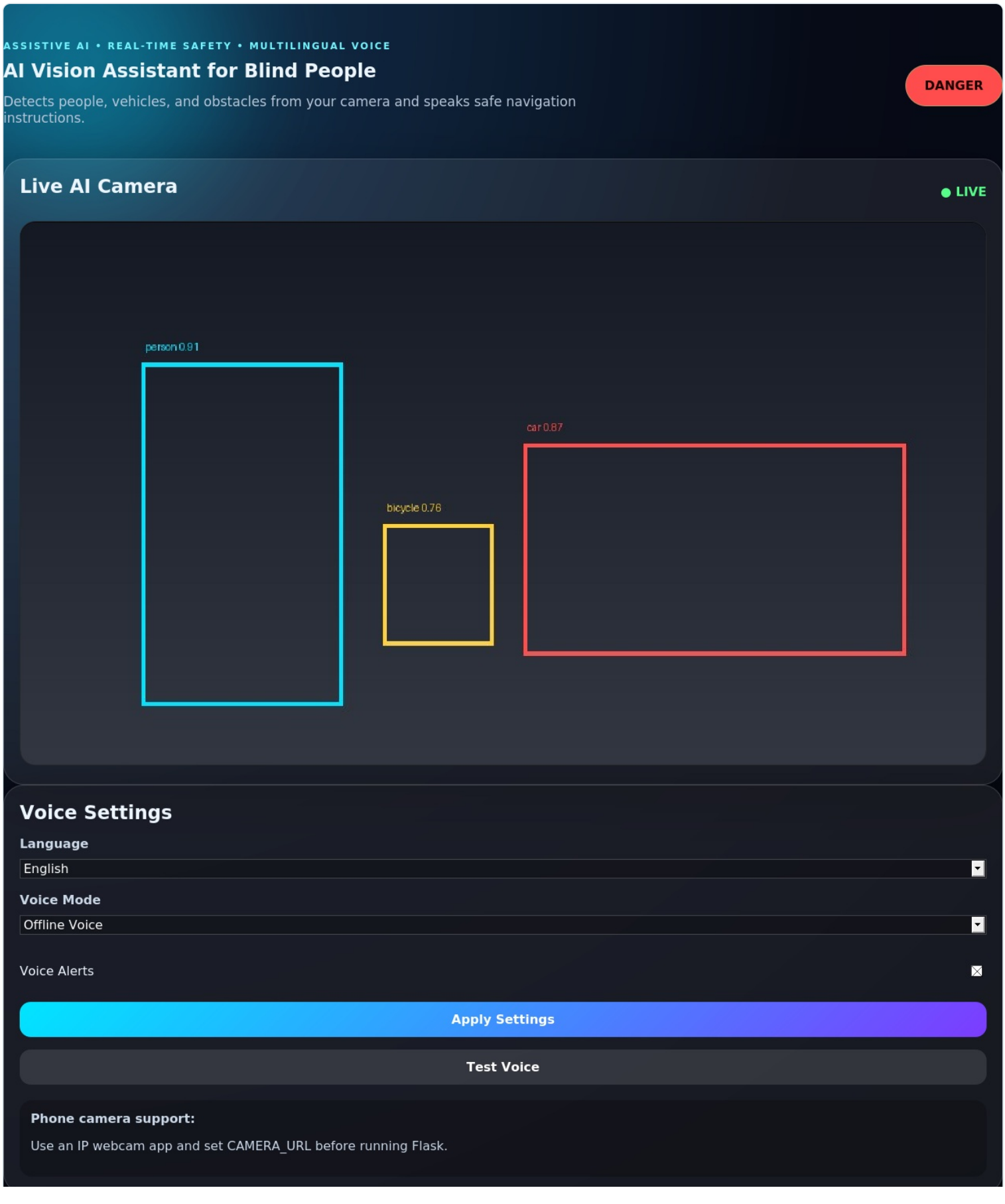


Figure 2 — Caution State. A single person is detected ahead at medium distance. The status pill turns yellow ("CAUTION"), and the spoken instruction advises the user to slow down.



Current Instruction

Warning: car approaching on the right. Stop and wait.

चेतावनी: दाईं ओर से कार आ रही है। रुकें और प्रतीक्षा करें।

Detected Objects

- person — 91%
- car — 87%
- bicycle — 76%

Figure 3 — Danger State. Multiple objects — a person, a car, and a bicycle — are detected simultaneously. Because a vehicle is centered and close, the system escalates to "DANGER" and instructs the user to stop and wait.

ASSISTIVE AI • REAL-TIME SAFETY • MULTILINGUAL VOICE

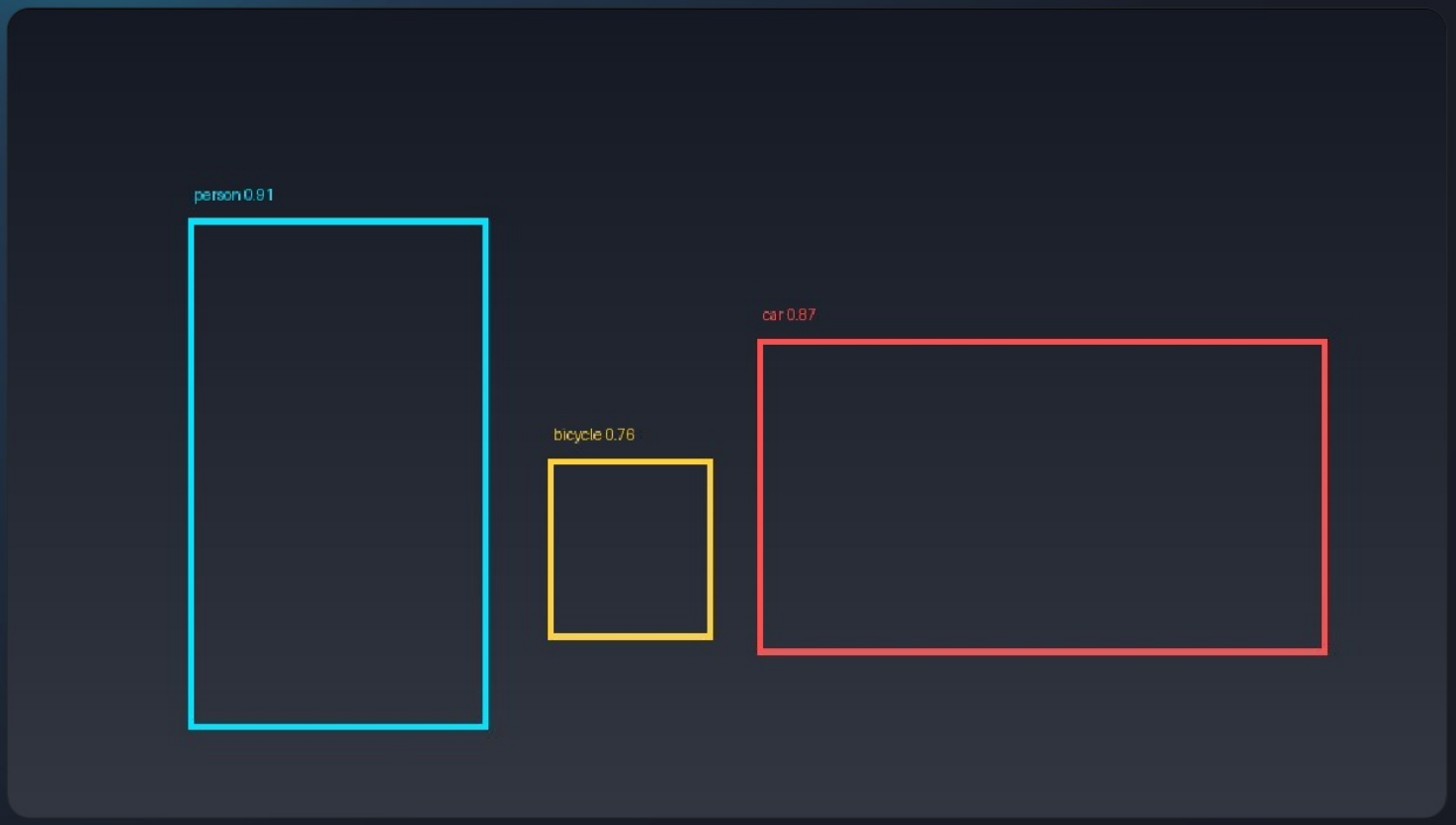
AI Vision Assistant for Blind People

Detects people, vehicles, and obstacles from your camera and speaks safe navigation instructions.

DANGER

Live AI Camera

● LIVE



Voice Settings

Language

Telugu

Voice Mode

Online Multilingual Voice

Voice Alerts ☒

Apply Settings

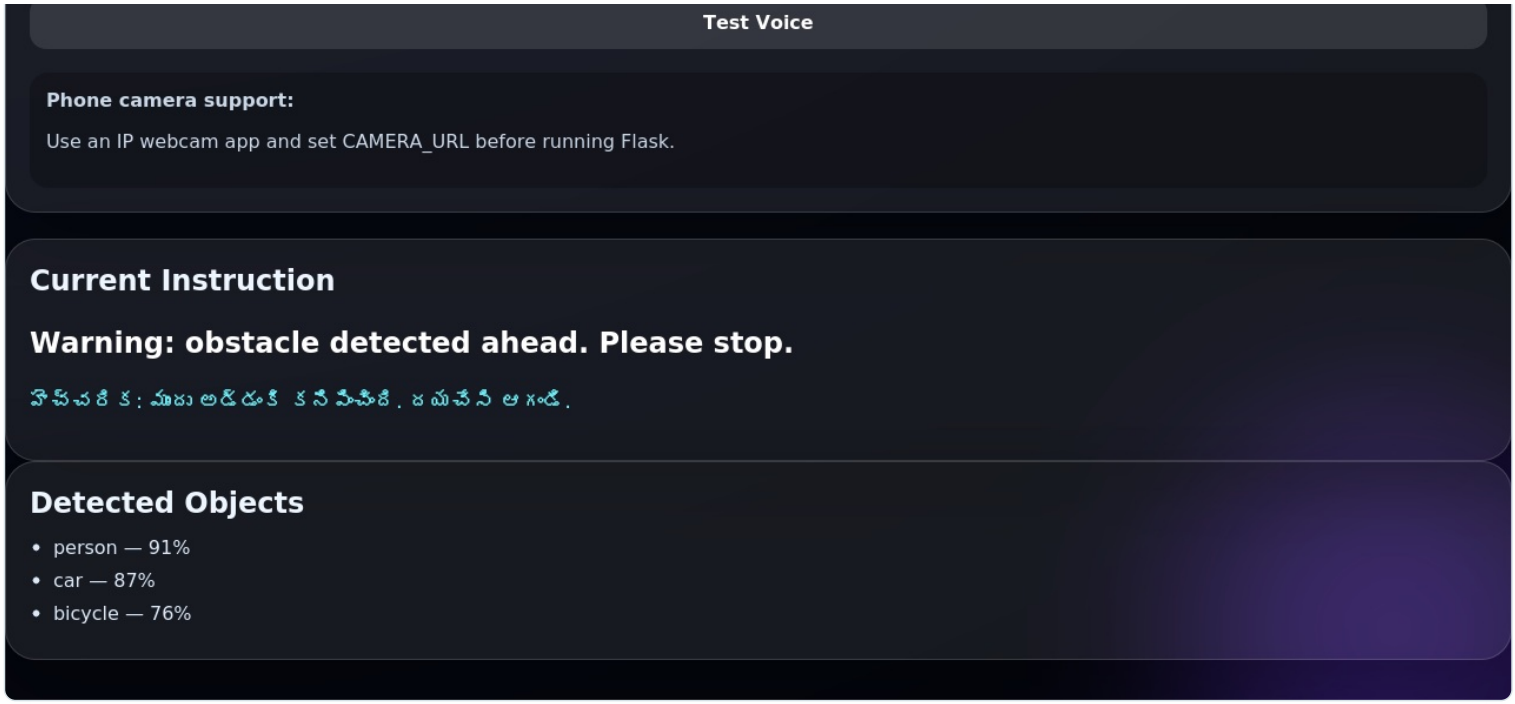


Figure 4 — Multilingual Voice Settings. The Voice Settings panel with Telugu selected as the output language and Online Multilingual Voice enabled; the spoken instruction is shown translated into Telugu alongside the original English text.

5. Future Scope

- Train a custom detection model for hazards not covered by general-purpose datasets, such as potholes, open manholes, staircases, open drainage, and zebra crossings.
- Add GPS-based outdoor navigation and route guidance.
- Add an emergency contact / SOS alert feature for high-risk situations.
- Integrate with dedicated smart-glasses hardware for a hands-free form factor.
- Add a vibration-feedback module as a silent, non-audio alert channel.
- Build a companion Android application.
- Add a MongoDB-backed dashboard for long-term journey history and analytics (the project already includes PyMongo as a dependency for this purpose).

6. Conclusion

This project demonstrates that a genuinely useful assistive vision system can be built entirely from open-source, off-the-shelf components: a laptop webcam, a pretrained YOLOv8 model, and standard text-to-speech engines. By combining real-time object detection with a simple direction-and-distance-based danger classification scheme, the system converts raw visual data into short, actionable spoken guidance — without requiring the user to interpret a screen. The addition of multilingual voice output further broadens accessibility, making the assistant usable by a wide range of visually impaired individuals regardless of their native language. While the current implementation is a proof-of-concept suited for a laptop or edge device, its modular design (separated detection, danger-logic, and voice components) makes it straightforward to extend toward the future scope items outlined above, moving it closer to a deployable, wearable assistive product.